1. Where in a max-heap might the smallest element reside, assuming that all elements are distinct?

2. Starting with the procedure MAX-HEAPIFY, write pseudocode for the procedure MIN-HEAPIFY(A, i), which performs the corresponding manipulation on a min-heap. How does the running time of MIN-HEAPIFY compare to that of MAX-HEAPIFY?

1. ∵父節點 ≥ 其子節點

∴最小的節點只會出現在葉節點中 2分

2. ①

```
MIN-HEAPIFY(A, i)
l ← LEFT(i)                                          1分
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] < A[i]                  1分
   then smallest ← l
   else smallest ← i
if r ≤ heap-size[A] and A[r] < A[smallest]           1分
   then smallest ← r
if smallest ≠ i                                      2分
   then exchange A[i] ↔ A[smallest]
      MIN-HEAPIFY(A, smallest)
```

② MAX-HEAPIFY 與 MIN-HEAPIFY 兩者時間複雜度相同皆為 $O(\lg n)$ 2分 1分

1. Where in a max-heap might the smallest element reside, assuming that all elements are distinct?

∵ parent node 必 ≥ child node

∴ 最小的元素一定會在葉節點

2. Starting with the procedure MAX-HEAPIFY, write pseudocode for the procedure MIN-HEAPIFY(A, i), which performs the corresponding manipulation on a min-heap. How does the running time of MIN-HEAPIFY compare to that of MAX-HEAPIFY?

```
MIN-HEAPIFY(A,i)
    l ← LEFT(i)
    r ← RIGHT(i)
    if l ≤ heap-size[A] and A[l]<A[i]
        then smallest ← l
        else smallest ← i
    if r ≤ heap-size[A] and A[r]<A[smallest]
        then smallest ← r
    if smallest ≠ i
        then exchange A[i] ↔ A[smallest]
             MIN-HEAPIFY(A, smallest)
```

先根左迴比 min(l,i)

再跟右迴比 min(smallest,r)

← 如果 child-node 較小

heapify:
先左，再右，最後看看要不要換

```
MAX-HEAIPY(A,i)
    l ← LEFT(i)
    r ← RIGHT(i)
    if l ≤ heapsize[A] and A[l] >A[i]
        then largest ← l
        else largest ← i
    if r ≤ heapsize[A] and A[r] > A[largest]
        then largest ← r
        else largest ← i
    if largest ≠ i
        then exchange A[i] ↔ A[largest]
        MAX-HEAPIFY(A, largest)
```

CH6 P.15

```
MAX-HEAPIFY(A, i)
1.  ℓ ← LEFT(i)  左子 (2i)
2.  r ← RIGHT(i) 右子(2i+1)
3.  if ℓ ≤ heap-size[A] and A[ℓ] > A[i]
4.  (有左子存在) then largest ← ℓ
5.       else largest ← i
                  } largest = max {l, i} 跟左子相比
6.  if r ≤ heap-size[A] and a[r] > A[largest]
7.  (有右子存在) then largest ← r
                  } largest = max {largest, r} 跟右子相比
8.  if largest ≠ i 如果子節点較大
9.       then exchange A[i] ↔A[largest]
10.           MAX-HEAPIFY (A, largest)
```

MAX-HEAP Insert

Insert：先多1空間、塞∞/-∞
最後call 增減！

MAX-HEAP-INSERT(A, *key*)

O(1) 1. *heap-size*[A] ← *heap-size*[A]+1 將 heap-size 加1
O(1) 2. A[*heap-size*[A]] ←−∞ 將最後一個放 -∞ (最小的值会在max-heap最下面)
O(lg n) 3. HEAP-INCREASE-KEY(A, *heap-size*[A], *key*)
將最後一個的key值從 -∞ 增加為 key值

(放-∞，確保所有值一定比此值大。)

MIN-HEAP Insert

MIN-HEAP-INSERT(A, key)
heap-size[A] ← heap-size[A] +1
A[heap-size[A]] ← ∞
HEAP-DECREASE-KEY(A, heap-size[A], key)

MAX-HEAP-INSERT(A, key)
heapsize[A] ← heapsize[A] + 1
A[heapsize[A]] ← -∞
HEAPIFY-INCRESE(A, heapsize[A], key)

MAX-HEAP Increase

HEAP-INCREASE-KEY (A, *i*, *key*)
1. **if** *key* < A[*i*] 檢查 key值真的有增加
2. **then error** "new key is smaller than current key"
3. A[*i*] ← *key*
4. **While** *i* > 1 and A[PARENT(*i*)] < A[*i*] 如果不是root，而且key值比父節点大，
5. **do** exchange A[*i*] ↔A[PARENT(*i*)] 與父節点做交換
6. *i* ← PARENT(*i*)

MAX-HEAP-INCREASE(A, key, i)
if key < A[i]
then error "new key smaller then current key"
while i>1 and A[Parent(i)]<A[i]
do exchange A[i] ↔ A[Parent(i)]
i ← Parent(i)

MIN-Heap Decrease

HEAP-DECREASE-KEY(A, i, key)
if key > A[i]
then error "new key is large then current key"
while i > 1 and A[Parent(i)] > A[i]
do exchange A[i] ↔ A[Parent(i)]
i ← Parent(i)

check Value 真的要減少

not root 且確實key比父來得小 →和父交換

heap Sort

```
HEAPSORT(A)
1.  BUILD-MAX-HEAP(A)
2.  for i ← length[A]  downto 2   從 n 往前做到 2
3.      do exchange A[1] ↔ A[i]
4.         heap-size[A] ← heap-size[A] −1
5.         MAX-HEAPIFY(A, 1)
```

交換
忽略 將 heap - size - 1
重新調整

3 step:
交換 exchange
忽略 ignore
重新調整 adjust